Unser Thema ist Mediahub

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:
Markus Tran
Abdulkerim Cufurovic

Betreuer:
Robert Reder

Leonding, 4. April 2025

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 4. April 2025 Markus Tran & Abdulkerim Cufurovic

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation und Problemstellung der Abo-Verwaltung 1

1.2 Zielsetzung der Diplomarbeit 1

1.3 Nutzen der Anwendung für Endanwender 1

1.4 Aufbau der Diplomarbeit 1

2 Ausgangssituation und Analyse **2**

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements 2

2.2 Probleme durch parallele Abos und doppelte Abbuchungen 2

2.3 Bestehende Lösungen und deren Schwächen 2

2.4 Abgrenzung zu Standard-Webprojekten 2

3 Fachlicher Hintergrund Abo- & Mediatheken-Management **3**

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten 3

3.2 Darstellung von Verfügbarkeiten von Filmen und Serien 3

3.3 Mehrwert einer zentralen Applikation für Nutzer 3

3.4 Datenschutz- und Sicherheitsanforderungen 3

4 Technologische Grundlagen **4**

4.1 Angular 4

4.2 Keycloak 5

4.3 Tailwind CSS 7

4.4 GitHub 8

5 Architektur und Systemdesign **10**

5.1 Gesamtarchitektur der Anwendung 10

5.2 Kommunikation zwischen Frontend und Backend 10

5.3 API-Integration externer Movie-Datenbanken 11

5.4 Sicherheits- und Datenschutzkonzept 11

6 Umsetzung der Anwendung **13**
6.1 Implementierung der Abo-Verwaltung 13
6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten 14
6.3 Verknüpfung von Filmen mit Streaming-Anbieter 15
6.4 Account- und Passwortverwaltung 16

7 Projektverlauf und Entscheidungen **19**
7.1 Projektplanung und Arbeitsweise 19
7.2 Technische und fachliche Herausforderungen 19
7.3 Entscheidungsfindung im Projektteam 19
7.4 Einbindung von Betreuern und Feedback 19
7.5 Reaktion auf Fragen und Kritik aus Präsentationen 19

8 Ergebnisse und Evaluation **20**
8.1 Erfüllung der gesetzten Ziele 20
8.2 Bewertung des Nutzens für Endanwender 20
8.3 Reflexion der eigenen Leistung 20

9 Ausblick **21**
9.1 Mögliche Erweiterungen der Anwendung 21
9.2 Zukunftspotenzial der Plattform 21

10 Zusammenfassung **22**

11 Quellen-, Literatur- und Abbildungsverzeichnis **23**
11.1 Quellen 23
11.2 Literatur 23
11.3 Abbildungsverzeichnis 23

Literaturverzeichnis **IV**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

Quellcodeverzeichnis **VII**

1 Einleitung

1.1 Motivation und Problemstellung der Abo-Verwaltung

Text ...

1.2 Zielsetzung der Diplomarbeit

Text ...

1.3 Nutzen der Anwendung für Endanwender

Text ...

1.4 Aufbau der Diplomarbeit

Text ...

2 Ausgangssituation und Analyse

2.1 Aktuelle Herausforderungen bei Streaming-Abonnements

Text ...

2.2 Probleme durch parallele Abos und doppelte Abbuchungen

Text ...

2.3 Bestehende Lösungen und deren Schwächen

Text ...

2.4 Abgrenzung zu Standard-Webprojekten

Text ...

3 Fachlicher Hintergrund Abo- & Mediatheken-Management

3.1 Zentrale Verwaltung von Abonnements und Laufzeiten

Text ...

3.2 Darstellung von Verfügbarkeiten von Filmen und Serien

Text ...

3.3 Mehrwert einer zentralen Applikation für Nutzer

Text ...

3.4 Datenschutz- und Sicherheitsanforderungen

Text ...

4 Technologische Grundlagen

4.1 Angular

Im Rahmen meines Projektteils lag der Schwerpunkt auf der Entwicklung des Frontends. Als technologische Grundlage wurde dafür Angular verwendet, da das Framework eine klare Struktur für moderne Webanwendungen bietet und zentrale Funktionen wie Routing, Komponentenarchitektur, Formularverarbeitung und HTTP-Kommunikation bereits integriert [1, 2, 3]. Für die Umsetzung des MedienAbonnementManagers war Angular besonders geeignet, da mehrere zusammenhängende Oberflächenbereiche entwickelt werden mussten, die dennoch in einer einheitlichen Anwendung miteinander verbunden bleiben sollten.

4.1.1 Rolle von Angular innerhalb der Anwendung

Angular übernimmt in der Anwendung die gesamte Darstellung und Interaktion der Benutzeroberfläche. Dazu zählen insbesondere die Startseite, die Filmsuche, die Detailansicht einzelner Filme, die Navigation innerhalb der Anwendung sowie die Oberfläche zur Verwaltung persönlicher Streaming-Abonnements. Der Fokus meines Projektteils lag darauf, diese Funktionen so umzusetzen, dass sie übersichtlich, wartbar und für Benutzer intuitiv bedienbar sind.

4.1.2 Architektur als Single-Page-Application

Das Frontend wurde als Single-Page-Application umgesetzt. Bei diesem Ansatz bleibt die Anwendung nach dem ersten Laden im Browser aktiv und Inhalte werden dynamisch aktualisiert, ohne dass bei jeder Aktion ein vollständiger Seitenneuaufbau erfolgen muss [4]. Diese Architektur verbessert die Benutzererfahrung deutlich, da Navigationsvorgänge zwischen Startseite, Filmsuche, Detailansicht und Providerverwaltung flüssig ablaufen. Gerade für eine interaktive Webanwendung mit mehreren Ansichten stellt dies einen wesentlichen Vorteil dar.

4.1.3 Begründung der Wahl von Angular gegenüber React

Für dieses Projekt wurde Angular gewählt, weil es ein vollständiges Framework mit klarer Struktur und fest integrierten Kernfunktionen darstellt [1, 5]. Im Vergleich zu React bietet Angular bereits standardisierte Lösungen für Routing, Formulare und HTTP-Kommunikation. Für den Frontend-Teil der Diplomarbeit war dies besonders hilfreich, da dadurch eine einheitliche Entwicklung möglich war und weniger zusätzliche Bibliotheken benötigt wurden. Außerdem erleichtert Angular durch seinen komponentenbasierten Aufbau die langfristige Wartung und Erweiterung der Anwendung.

4.1.4 Komponentenstruktur der Anwendung

Die Benutzeroberfläche wurde in mehrere eigenständige Angular-Komponenten gegliedert. Dazu gehören unter anderem die Hauptseite, die Filmübersicht, die Detailansicht eines Films, die Providerverwaltung, die Navigationsleiste sowie die Ansichten für Login und Registrierung. Diese Aufteilung ermöglicht eine saubere Trennung einzelner Funktionsbereiche. Änderungen an einer Komponente können dadurch umgesetzt werden, ohne andere Bereiche der Anwendung unnötig zu beeinflussen.

4.1.5 Data Binding und Services zur Zustandsverwaltung

Ein wesentlicher Bestandteil der Frontend-Entwicklung war die Verknüpfung von Benutzereingaben mit den dargestellten Daten. Angular stellt dafür Mechanismen wie Data Binding und Formularunterstützung bereit [3]. Im Projekt wurden außerdem eigene Services eingesetzt, um Datenzugriffe zu kapseln und die Kommunikation mit den bereitgestellten Schnittstellen zu organisieren. Dadurch bleibt die Oberflächenlogik übersichtlich und die einzelnen Komponenten konzentrieren sich auf Darstellung und Interaktion.

4.2 Keycloak

Für die Anmeldung und Registrierung der Benutzer wurde im Frontend Keycloak eingebunden. Dadurch musste keine eigene Login-Oberfläche mit selbst implementierter Passwortlogik entwickelt werden. Stattdessen wurde eine etablierte Lösung für Authentifizierung verwendet, die das Frontend mit dem Anmeldesystem verbindet [6, 7]. Aus

Sicht meines Projektteils war vor allem die Einbindung in die Benutzeroberfläche und die Steuerung geschützter Frontend-Bereiche relevant.

4.2.1 Grundlagen der Authentifizierung aus Frontend-Sicht

Im Frontend dient die Authentifizierung dazu, zwischen öffentlichen und geschützten Bereichen der Anwendung zu unterscheiden. Öffentliche Inhalte wie die Filmübersicht können allgemein angezeigt werden, während persönliche Bereiche wie die Verwaltung eigener Abonnements nur angemeldeten Benutzern zugänglich sind. Das Frontend reagiert damit auf den Authentifizierungsstatus und passt Navigation sowie verfügbare Funktionen entsprechend an.

4.2.2 Ablauf des Login- und Token-Prozesses

Beim Login wird der Benutzer aus dem Frontend heraus an Keycloak weitergeleitet. Nach erfolgreicher Anmeldung kehrt er in die Anwendung zurück und kann geschützte Funktionen nutzen [6]. Für das Frontend ist dabei entscheidend, dass der Benutzerstatus erkannt und für weitere Anfragen verwendet wird. Auf diese Weise können geschützte Ansichten nur dann geöffnet werden, wenn eine gültige Anmeldung vorliegt.

4.2.3 Rollen und Zugriff auf geschützte Bereiche

Neben der reinen Anmeldung spielt auch die Zugriffskontrolle auf bestimmte Ansichten eine Rolle. Im Frontend wird geprüft, ob ein Benutzer auf geschützte Bereiche wie die Providerverwaltung zugreifen darf. Dadurch kann die Navigation benutzerfreundlich gestaltet werden, indem nicht erlaubte Seiten entweder verborgen oder nur nach erfolgreicher Anmeldung erreichbar gemacht werden.

4.2.4 Nutzen für Sicherheit und Benutzerfreundlichkeit

Die Verwendung von Keycloak verbessert sowohl die Sicherheit als auch die Benutzerfreundlichkeit. Da keine eigene Passwortverwaltung entwickelt werden musste, konnte das Risiko unsicherer Eigenimplementierungen reduziert werden. Gleichzeitig wirkt der Login-Prozess für Benutzer konsistent, da Registrierung, Anmeldung und Abmeldung zentral über ein spezialisiertes System gesteuert werden.

4.3 Tailwind CSS

Für die Gestaltung der Benutzeroberfläche wurde Tailwind CSS eingesetzt. Tailwind ist ein CSS-Framework, das nach dem Utility-First-Prinzip arbeitet und dadurch eine schnelle und flexible Entwicklung moderner Benutzeroberflächen ermöglicht [8, 9]. Im Frontend des MedienAbonnementManagers wurde Tailwind genutzt, um Layout, Abstände, Farben, Formulare und Kartenansichten effizient zu gestalten.

4.3.1 Utility-First-Ansatz

Der Utility-First-Ansatz bedeutet, dass viele kleine CSS-Klassen direkt in den Templates verwendet werden, um konkrete Gestaltungseigenschaften festzulegen [8]. Dazu zählen unter anderem Abstände, Größen, Farben, Rahmen oder Typografie. Für das Projekt hatte dies den Vorteil, dass Oberflächen schnell angepasst und konsistent gestaltet werden konnten, ohne für jede Änderung umfangreiche eigene CSS-Strukturen anlegen zu müssen.

4.3.2 Gestaltung der Benutzeroberfläche

Tailwind CSS wurde besonders für interaktive Bereiche wie die Providerverwaltung, Formulare, Suchfelder, Buttons und Kartenansichten eingesetzt. Dadurch konnte eine moderne und übersichtliche Oberfläche umgesetzt werden, die die verschiedenen Funktionsbereiche visuell voneinander trennt. Vor allem in der Abo-Verwaltung war eine klare Darstellung wichtig, damit Benutzer Anbieter, Status und Eingabefelder schnell erfassen können.

4.3.3 Integration von Tailwind CSS in Angular

Die Einbindung von Tailwind CSS in das Angular-Projekt erfolgte direkt über die Konfiguration des Frontends. Dadurch standen die Utility-Klassen in allen Komponenten zur Verfügung. Diese enge Integration war für die praktische Frontend-Entwicklung besonders hilfreich, da Layout und Funktionalität parallel umgesetzt werden konnten.

4.3.4 Beitrag zur Benutzerfreundlichkeit

Ein zentrales Ziel des Frontends war eine intuitive und ansprechende Bedienung. Die Gestaltung mit Tailwind CSS unterstützt dieses Ziel, indem wichtige Elemente klar hervorgehoben und logisch angeordnet werden. Eine gut strukturierte Oberfläche trägt wesentlich dazu bei, dass Benutzer die Anwendung leichter verstehen und effizient verwenden können.

4.4 GitHub

GitHub wurde im Projekt als Plattform für Versionsverwaltung und Zusammenarbeit genutzt. Für meinen Frontend-Teil war dies insbesondere wichtig, um Änderungen an Komponenten, Styles und Routing nachvollziehbar zu dokumentieren und in einem strukturierten Entwicklungsprozess festzuhalten [10]. Dadurch konnten einzelne Entwicklungsschritte sauber voneinander getrennt und schrittweise weiterentwickelt werden.

4.4.1 Versionsverwaltung im Frontend

Die Frontend-Entwicklung umfasste mehrere aufeinander aufbauende Änderungen, beispielsweise an der Filmsuche, an der Navigationsstruktur, an der Provideransicht und am visuellen Design. Durch die Versionsverwaltung konnten diese Schritte einzeln dokumentiert und bei Bedarf auf frühere Stände zurückgeführt werden. Dies erhöhte die Nachvollziehbarkeit und reduzierte das Risiko, funktionierende Zwischenstände zu verlieren.

4.4.2 Branching und strukturierte Weiterentwicklung

Im Projekt wurden verschiedene Branches verwendet, um neue Änderungen getrennt vom Hauptstand weiterzuentwickeln. Für die Frontend-Arbeit war dies besonders hilfreich, wenn größere Überarbeitungen an der Benutzeroberfläche oder an der Logik einzelner Ansichten vorgenommen wurden. So konnten neue Ideen und Anpassungen zunächst isoliert entwickelt werden, bevor sie in einen stabileren Stand übernommen wurden.

4.4.3 Nachvollziehbarkeit der eigenen Arbeit

Die Commit-Historie dokumentiert die schrittweise Weiterentwicklung des Frontends, etwa bei der Überarbeitung der Filmsuche, beim Ausbau der Trending-Ansichten oder bei der Verbesserung der Benutzeroberfläche der Abo-Verwaltung. Für die Diplomarbeit ist dies hilfreich, da der eigene Beitrag am Frontend dadurch nachvollziehbar beschrieben werden kann.

5 Architektur und Systemdesign

5.1 Gesamtarchitektur der Anwendung

Die Gesamtarchitektur der Anwendung folgt einem klar getrennten Aufbau zwischen Benutzeroberfläche und serverseitiger Verarbeitung. Der Schwerpunkt meines Projektteils lag auf dem Frontend, das mit Angular als Single-Page-Application umgesetzt wurde. Das Frontend übernimmt die Darstellung der Inhalte, die Navigation zwischen den Ansichten sowie die Interaktion mit den Benutzern. Die eigentliche Datenverarbeitung erfolgt über bereitgestellte REST-Schnittstellen, die vom Frontend angesprochen werden. Durch diese Trennung konnte die Benutzeroberfläche unabhängig von der serverseitigen Implementierung entwickelt und erweitert werden.

5.2 Kommunikation zwischen Frontend und Backend

Die Kommunikation zwischen Frontend und Backend erfolgt über HTTP-basierte REST-Aufrufe. Im Frontend wurden dafür eigene Angular-Services verwendet, um Daten strukturiert von den Schnittstellen zu laden und an die Komponenten weiterzugeben [3]. Dieses Vorgehen hat den Vorteil, dass die Komponenten selbst möglichst wenig Logik für Datenzugriffe enthalten und sich stärker auf Darstellung und Interaktion konzentrieren können.

Im Projekt wurden insbesondere Services für Filmdaten, Providerdaten und die Authentifizierungsanbindung eingesetzt. Dadurch konnte die Anwendung Suchanfragen, Detailansichten und benutzerspezifische Inhalte in einer einheitlichen Struktur verarbeiten. Für das Frontend war dabei vor allem wichtig, dass externe Daten zuverlässig geladen, zwischengespeichert und in geeigneter Form in der Oberfläche dargestellt werden.

Bei benutzerspezifischen Daten reicht ein normaler HTTP-Aufruf jedoch nicht aus. Solche Daten können nur dann vom Backend geladen werden, wenn das Frontend ein gültiges Bearer-Token mitsendet. Dieses Token wird nach erfolgreicher Anmeldung

über Keycloak bereitgestellt und bei geschützten Anfragen im Authorization-Header übertragen. Auf diese Weise kann das Backend prüfen, ob der Benutzer authentifiziert ist und Zugriff auf die angeforderten Daten besitzt [6, 7].

5.3 API-Integration externer Movie-Datenbanken

Ein wichtiger Bestandteil der Anwendung ist die Darstellung von Filminformationen und Streaming-Anbietern. Aus Sicht des Frontends bestand die Aufgabe darin, diese Informationen benutzerfreundlich aufzubereiten und in unterschiedlichen Ansichten darzustellen. Dazu gehören insbesondere die Filmübersicht, die Detailansicht einzelner Filme sowie die Anzeige verfügbarer Anbieter.

Die Filmdaten werden über bereitgestellte Schnittstellen geladen und in Angular-Komponenten eingebunden. Für die Benutzeroberfläche war dabei entscheidend, dass Suchergebnisse übersichtlich dargestellt, Detailinformationen verständlich aufbereitet und Navigationswechsel zwischen Listen- und Detailansicht flüssig umgesetzt werden. Da die Anwendung als Single-Page-Application realisiert wurde, konnte dieser Wechsel ohne vollständigen Seitenneuaufbau erfolgen [4].

5.4 Sicherheits- und Datenschutzkonzept

Aus Frontend-Sicht spielt das Sicherheits- und Datenschutzkonzept vor allem bei der Zugriffskontrolle auf geschützte Bereiche eine Rolle. Öffentliche Inhalte der Anwendung können ohne Anmeldung aufgerufen werden, während persönliche Funktionen wie die Verwaltung eigener Streaming-Abonnements nur nach erfolgreicher Authentifizierung zugänglich sind. Für diesen Zweck wurde Keycloak in das Frontend integriert [6, 7].

Ein wichtiger Bestandteil dieses Konzepts ist, dass personenbezogene oder benutzerspezifische Daten nicht öffentlich abrufbar sind. Das Backend stellt diese Informationen nur dann bereit, wenn bei der Anfrage ein gültiges Bearer-Token übertragen wird. Das Frontend übernimmt dabei die Aufgabe, den Authentifizierungsstatus zu verwalten und das Token bei geschützten Requests mitzuschicken. Dadurch wird sichergestellt, dass persönliche Inhalte nur autorisierten Benutzern angezeigt werden [6].

Die Angular-Anwendung erkennt den Authentifizierungsstatus der Benutzer und steuert darauf basierend den Zugriff auf geschützte Routen. Dadurch wird sichergestellt,

dass sensible benutzerspezifische Inhalte nicht ohne Anmeldung angezeigt werden. Gleichzeitig verbessert diese Struktur die Benutzerfreundlichkeit, da sich öffentliche und persönliche Bereiche klar voneinander trennen lassen. Die eigentliche Verwaltung von Zugangsdaten erfolgt dabei nicht im Frontend selbst, sondern über das externe Authentifizierungssystem.

6 Umsetzung der Anwendung

6.1 Implementierung der Abo-Verwaltung

Ein wesentlicher Bestandteil meines Projektteils war die Umsetzung der Benutzeroberfläche zur Verwaltung persönlicher Streaming-Abonnements. Ziel war es, eine übersichtliche und einfach bedienbare Ansicht zu entwickeln, in der Benutzer ihre aktiven Anbieter verwalten und Änderungen direkt speichern können. Die technische Umsetzung erfolgte im Angular-Frontend mit einer eigenständigen Komponente für die Providerverwaltung sowie zugehörigen Services für die Kommunikation mit dem Backend [1, 3].

6.1.1 Aufbau der Provider-Verwaltung

Die Provider-Verwaltung wurde als eigener geschützter Bereich innerhalb der Angular-Anwendung umgesetzt. Benutzer können dort Streaming-Anbieter laden, durchsuchen und auswählen. Zusätzlich ist erkennbar, welche Anbieter bereits im Profil gespeichert sind. Diese Struktur verbessert die Übersichtlichkeit und ermöglicht eine klare Trennung zwischen allgemeinen Filmfunktionen und persönlichen Abo-Daten.

6.1.2 Bearbeitung und Speicherung von Abonnements

In der Oberfläche können Benutzer Anbieter als aktiv markieren und zusätzliche Angaben wie Abrechnungszyklus und Datum der letzten Abbuchung festlegen. Diese Eingaben werden im Frontend verarbeitet und anschließend gesammelt an das Backend übermittelt. Die Verwendung von Formularlogik und Datenbindung in Angular erleichtert dabei die direkte Verknüpfung zwischen Eingabefeldern und angezeigten Daten [3]. Dadurch konnten Änderungen in der Oberfläche unmittelbar sichtbar gemacht werden.

6.1.3 Filter- und Suchfunktionen in der Oberfläche

Damit die Verwaltung vieler Anbieter benutzerfreundlich bleibt, wurde die Oberfläche um Such- und Filterfunktionen ergänzt. Benutzer können nach Anbietern suchen oder sich nur bereits aktivierte Abonnements anzeigen lassen. Diese Interaktionsmöglichkeiten verbessern die Nutzbarkeit der Anwendung deutlich, da relevante Einträge schneller gefunden und bearbeitet werden können.

6.2 Darstellung von Laufzeiten und Kündigungszeitpunkten

Neben der Auswahl von Streaming-Anbietern war auch die Darstellung zeitlicher Informationen ein wichtiger Teil der Abo-Verwaltung. Aus Frontend-Sicht ging es dabei vor allem darum, Eingabemöglichkeiten für abrechnungsrelevante Daten bereitzustellen und diese verständlich in die Oberfläche einzubinden. Die Benutzer sollen dadurch ihre gespeicherten Abonnements besser nachvollziehen können.

6.2.1 Erfassung abrechnungsrelevanter Daten

Für jedes aktive Abonnement können im Frontend Informationen wie Abrechnungszyklus und Datum der letzten Abbuchung erfasst werden. Diese Werte werden in Formularfeldern dargestellt und vom Benutzer bearbeitet. Die Oberfläche unterstützt dadurch nicht nur die Auswahl eines Anbieters, sondern auch die strukturierte Verwaltung der wichtigsten Abo-Metadaten.

6.2.2 Grenzen des aktuellen Umsetzungsstandes

Im aktuellen Projektstand liegt der Schwerpunkt auf der Erfassung und Speicherung dieser Daten. Eine vollständige Logik zur Berechnung von Kündigungsfristen, Erinnerungen oder automatischen Benachrichtigungen wurde im Frontend noch nicht vollständig ausgebaut. Für die schriftliche Arbeit ist es daher sinnvoll, zwischen der bereits umgesetzten Datenerfassung und möglichen zukünftigen Erweiterungen zu unterscheiden.

6.3 Verknüpfung von Filmen mit Streaming-Anbieter

Ein zentraler Mehrwert der Anwendung besteht darin, dass Filmrecherche und Abo-Verwaltung in einer Oberfläche zusammengeführt werden. Im Frontend wurde dafür eine Benutzeroberfläche entwickelt, in der Benutzer Filme suchen, Trending-Inhalte ansehen und Detailinformationen zu einzelnen Filmen aufrufen können. Die Verknüpfung mit Streaming-Anbietern ermöglicht es, Filminformationen nicht isoliert, sondern im Zusammenhang mit den eigenen Abonnements zu betrachten.

6.3.1 Umsetzung der Filmsuche

Die Filmsuche wurde als eigener Bereich im Frontend umgesetzt. Benutzer können nach Filmtiteln suchen und erhalten eine Liste passender Ergebnisse, die dynamisch geladen und angezeigt werden. Da das Frontend als Single-Page-Application realisiert wurde, bleiben Suchanfragen und Seitenwechsel innerhalb derselben Anwendung und wirken dadurch deutlich flüssiger [4, 2].

6.3.2 Darstellung von Trending-Filmen

Zusätzlich zur klassischen Suche wurde eine Ansicht für Trending-Filme integriert. Dadurch können Benutzer aktuelle oder populäre Inhalte entdecken, ohne gezielt nach einem Titel suchen zu müssen. Diese Funktion erweitert die Anwendung um einen explorativen Aspekt und erhöht den praktischen Nutzen des Frontends.

6.3.3 Detailansicht einzelner Filme

Für einzelne Filme wurde eine eigene Detailansicht umgesetzt. Dort werden weiterführende Informationen wie Titel, Beschreibung, Erscheinungsjahr, Laufzeit, Genres und weitere Metadaten dargestellt. Die Aufbereitung dieser Daten im Frontend war wichtig, damit die Benutzer nicht nur Suchergebnisse sehen, sondern Inhalte auch genauer beurteilen können.

6.3.4 Verbindung zur Anbieteranzeige

Ein besonderer Fokus lag auf der Darstellung von Streaming-Anbietern im Zusammenhang mit Filminformationen. Dadurch kann die Anwendung anzeigen, auf welchen Plattformen ein Film verfügbar ist. Aus Sicht des Frontends bestand die Aufgabe vor allem darin, diese Informationen übersichtlich darzustellen und sinnvoll in die Filmansichten zu integrieren. Diese Verbindung zwischen Inhalt und Verfügbarkeit stellt den eigentlichen Kernnutzen des Projekts dar.

6.4 Account- und Passwortverwaltung

Die Benutzerverwaltung wurde im Frontend nicht als Eigenlösung umgesetzt, sondern über die Integration von Keycloak realisiert. Dadurch konnte auf eine eigene Passwortspeicherung verzichtet werden, während gleichzeitig ein professioneller und sicherer Anmeldeprozess in die Anwendung eingebunden wurde [6, 7]. Für meinen Projektteil war vor allem relevant, wie sich Login, Registrierung und geschützte Routen in die Angular-Anwendung integrieren lassen.

6.4.1 Einbindung von Login und Registrierung

Im Frontend wurden Login- und Registrierungsfunktionen so eingebunden, dass Benutzer bei Bedarf an das Keycloak-System weitergeleitet werden. Nach erfolgreicher Anmeldung kehren sie in die Anwendung zurück und können geschützte Bereiche verwenden. Dadurch bleibt der Authentifizierungsprozess klar vom restlichen Frontend getrennt und dennoch benutzerfreundlich in die Anwendung eingebettet.

6.4.2 Geschützte Bereiche im Frontend

Bestimmte Bereiche, insbesondere die Verwaltung persönlicher Abonnements, sind nur für angemeldete Benutzer zugänglich. Das Frontend prüft dafür den Authentifizierungsstatus und steuert den Zugriff auf geschützte Routen. Dadurch wird verhindert, dass persönliche Funktionen ohne gültige Anmeldung aufgerufen werden können.

6.4.3 Verwendung von Bearer-Token bei geschützten Requests

Ein wichtiger technischer Punkt der Umsetzung war, dass benutzerspezifische Daten nicht ohne Authentifizierung geladen werden können. Wenn das Frontend persönliche Informationen vom Backend anfordert, muss ein gültiges Bearer-Token mitgesendet werden. Dieses Token wird nach erfolgreicher Anmeldung über Keycloak bereitgestellt und bei geschützten HTTP-Anfragen übertragen [6]. Erst dadurch kann das Backend prüfen, ob der Benutzer berechtigt ist, auf die jeweiligen Daten zuzugreifen.

6.4.4 Herausforderungen bei der Integration von Keycloak

Die Integration von Keycloak in Angular war mit mehreren Herausforderungen verbunden. Dazu gehörten insbesondere die korrekte Weiterleitung nach dem Login, die Behandlung geschützter Routen sowie die sichere Übergabe des Bearer-Tokens bei Anfragen an das Backend. Diese Punkte waren im Frontend besonders relevant, da die Anwendung nur dann benutzerfreundlich funktioniert, wenn Anmeldung, Rückleitung und Zugriffsschutz zuverlässig zusammenspielen. Die Auseinandersetzung mit diesen Problemen war daher ein wichtiger Teil meines praktischen Projektbeitrags.

6.4.5 Analyse bestehender Lösungen

Im Zusammenhang mit Benutzerkonten stellte sich die Frage, ob zusätzlich eine eigene Passwortverwaltung innerhalb der Anwendung umgesetzt werden sollte. Bei der Betrachtung bestehender Lösungen wie Bitwarden oder 1Password wird jedoch deutlich, dass sich professionelle Passwortverwaltung auf Sicherheit, Verschlüsselung und geräteübergreifende Synchronisation spezialisiert [7]. Diese Zielsetzung unterscheidet sich klar vom Schwerpunkt des MedienAbonnementManagers.

6.4.6 Sicherheits- und Datenschutzbetrachtung

Eine selbst entwickelte Verwaltung von Passwörtern oder besonders sensiblen Zugangsdaten hätte hohe Anforderungen an Datenschutz und Sicherheit gestellt. Da der Fokus meines Projektteils auf der Frontend-Entwicklung lag, wurde bewusst auf eine eigene Passwortlösung verzichtet. Stattdessen war es sinnvoller, ein bestehendes Authentifizie-

rungssystem einzubinden und die Anwendung auf ihre eigentlichen Kernfunktionen zu konzentrieren.

6.4.7 Entscheidung: Eigenlösung vs. Integration

Die Entscheidung fiel daher zugunsten einer Integration von Keycloak und gegen eine selbst entwickelte Passwortverwaltung. Diese Lösung reduziert die Komplexität im Frontend, verbessert die Sicherheit und ermöglicht eine klarere Trennung zwischen Benutzeroberfläche und Authentifizierungslogik. Für das Projekt war diese Entscheidung sowohl technisch als auch organisatorisch sinnvoll.

7 Projektverlauf und Entscheidungen

7.1 Projektplanung und Arbeitsweise

7.2 Technische und fachliche Herausforderungen

7.3 Entscheidungsfindung im Projektteam

7.4 Einbindung von Betreuern und Feedback

7.5 Reaktion auf Fragen und Kritik aus Präsentationen

8 Ergebnisse und Evaluation

8.1 Erfüllung der gesetzten Ziele

8.2 Bewertung des Nutzens für Endanwender

8.3 Reflexion der eigenen Leistung

9 Ausblick

9.1 Mögliche Erweiterungen der Anwendung

9.2 Zukunftspotenzial der Plattform

10 Zusammenfassung

11 Quellen-, Literatur- und Abbildungsverzeichnis

11.1 Quellen

11.2 Literatur

11.3 Abbildungsverzeichnis

Literaturverzeichnis

[1] Angular Team, „What is Angular?" https://angular.dev/overview, 2026, Zugriff am 19.03.2026.

[2] ——, „Angular Routing," https://angular.dev/guide/routing, 2026, Zugriff am 19.03.2026.

[3] ——, „Forms," https://angular.dev/guide/forms, 2026, Zugriff am 19.03.2026.

[4] Mozilla Developer Network, „Single-page application," https://developer.mozilla.org/en-US/docs/Glossary/SPA, 2026, Zugriff am 19.03.2026.

[5] A. Freeman, *Pro Angular*. Apress, 2019.

[6] Keycloak Team, „Keycloak Documentation," https://www.keycloak.org/documentation, 2026, Zugriff am 19.03.2026.

[7] ——, „Keycloak," https://www.keycloak.org/, 2026, Zugriff am 19.03.2026.

[8] Tailwind Labs, „Tailwind CSS Documentation," https://tailwindcss.com/docs, 2026, Zugriff am 19.03.2026.

[9] ——, „Tailwind CSS," https://tailwindcss.com/, 2026, Zugriff am 19.03.2026.

[10] GitHub, Inc., „Git workflows," https://docs.github.com/en/get-started/getting-started-with-git/git-workflows, 2026, Zugriff am 19.03.2026.

Abbildungsverzeichnis

Tabellenverzeichnis

Quellcodeverzeichnis